

NYC Crash Data Exploration

Aansh Jha

```
from pathlib import Path
import json

import matplotlib.pyplot as plt
from matplotlib.patches import Polygon
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

DATA_DIR = Path("data")
CRASH_FILE = DATA_DIR / "nyc_crashes_2024-06-30_to_2024-07-06.csv"
FEATHER_FILE = DATA_DIR / "derived" / "nyc_crashes_2024-06-30_to_2024-07-06.feather"
FEATHER_FILE.parent.mkdir(exist_ok=True)
```

Load and compare storage formats

```
df = pd.read_csv(
    CRASH_FILE,
    parse_dates=["crash_date"],
    dtype={"crash_time": "string", "zip_code": "string"},
)
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(r"^[a-z0-9]+", "_", regex=True)
    .str.replace(r"(?<=code)(\d)$", r"_\1", regex=True)
    .str.strip("_")
)

df.to_feather(FEATHER_FILE)
reloaded = pd.read_feather(FEATHER_FILE)

print(f"Rows and columns: {df.shape}")
print(f"CSV size: {CRASH_FILE.stat().st_size / 1024**2:.2f} MB")
```

```
print(f"Feather size: {FEATHER_FILE.stat().st_size / 1024**2:.2f} MB")
print(f"Feather round-trip preserved shape: {reloaded.shape == df.shape}")
```

Rows and columns: (1676, 29)
 CSV size: 0.39 MB
 Feather size: 0.17 MB
 Feather round-trip preserved shape: True

1. Missing geocode data by borough

Values outside a conservative NYC bounding box, including (0, 0), are treated as missing geocodes.

```
df["missing_geocode"] = ~(
    df["latitude"].between(40.45, 40.95)
    & df["longitude"].between(-74.30, -73.65)
)

geocode_table = pd.crosstab(df["borough"], df["missing_geocode"])
display(geocode_table)

chi2, p_value, degrees_of_freedom, expected = chi2_contingency(geocode_table)
print(f"Chi-square statistic: {chi2:.3f}")
print(f"Degrees of freedom: {degrees_of_freedom}")
print(f"P-value: {p_value:.4f}")
print(
    "Conclusion: "
    + (
        "reject the null hypothesis of independence."
        if p_value < 0.05
        else "do not reject the null hypothesis of independence."
    )
)
```

missing_geocode borough	False	True
BRONX	185	4
BROOKLYN	413	3
MANHATTAN	210	3
QUEENS	334	2
STATEN ISLAND	44	1

Chi-square statistic: 3.890
 Degrees of freedom: 4

P-value: 0.4211

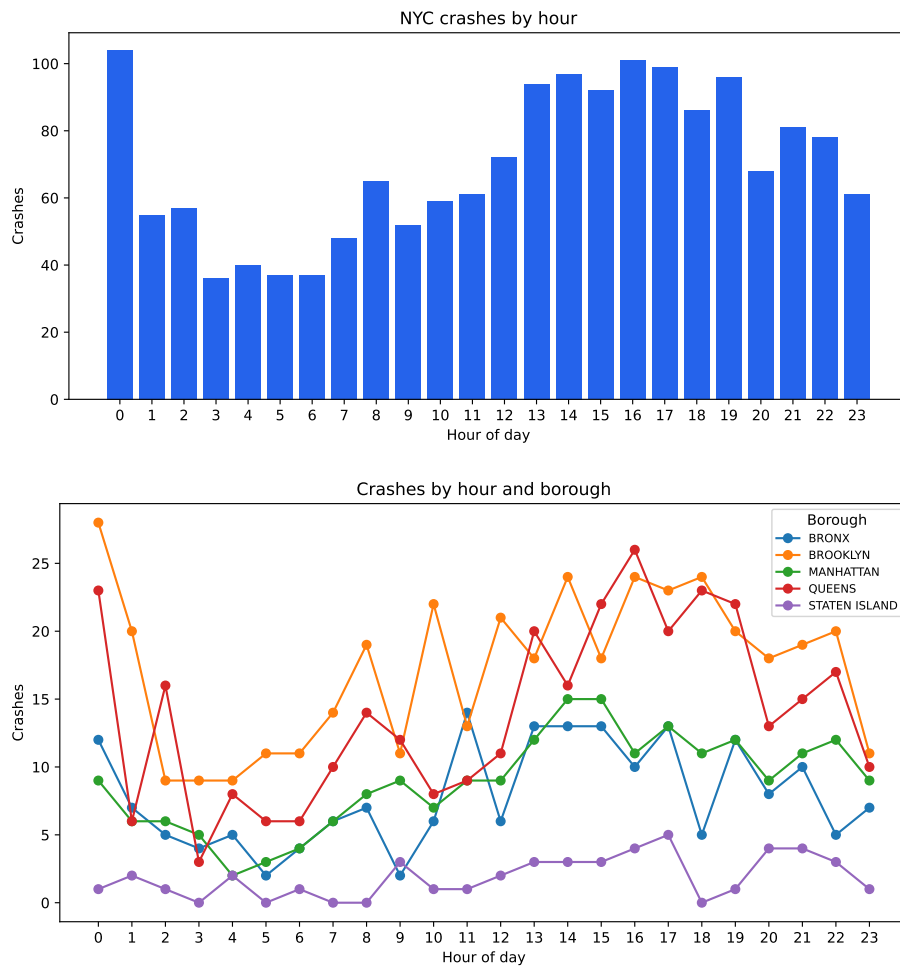
Conclusion: do not reject the null hypothesis of independence.

2. Crash counts by hour

```
df["crash_hour"] = pd.to_datetime(
    df["crash_time"], format="%H:%M", errors="coerce"
).dt.hour

hour_counts = df["crash_hour"].value_counts().reindex(range(24), fill_value=0)
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.bar(hour_counts.index, hour_counts.values, color="#2563eb")
ax.set(
    xlabel="Hour of day",
    ylabel="Crashes",
    title="NYC crashes by hour",
    xticks=range(24),
)
fig.tight_layout()
plt.show()

borough_hour = pd.crosstab(df["crash_hour"], df["borough"]).reindex(
    range(24), fill_value=0
)
ax = borough_hour.plot(figsize=(9, 5), marker="o")
ax.set(xlabel="Hour of day", ylabel="Crashes", title="Crashes by hour and borough")
ax.set_xticks(range(24))
ax.legend(title="Borough", fontsize=8)
plt.tight_layout()
plt.show()
```



3. Crash locations on a map of NYC

The boundary layer is stored with the project, so the map renders without a live tile service.

```
with open(DATA_DIR / "nyc_borough_boundaries.geojson", encoding="utf-8") as handle:
    borough_geojson = json.load(handle)

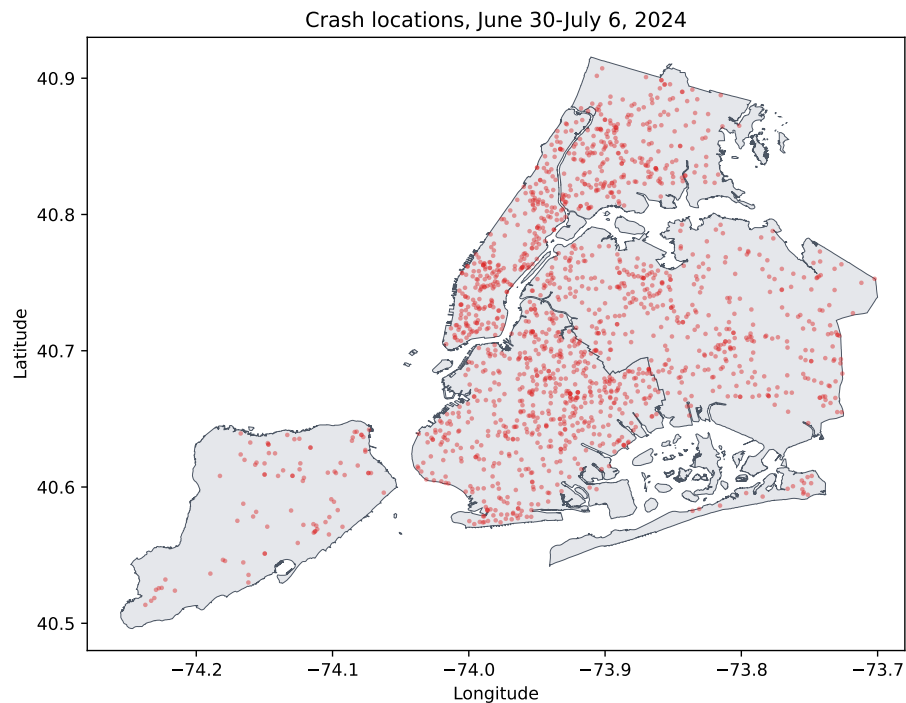
fig, ax = plt.subplots(figsize=(7.5, 8))
for feature in borough_geojson["features"]:
    geometry = feature["geometry"]
    polygons = (
        [geometry["coordinates"]]
        if geometry["type"] == "Polygon"
```

```

        else geometry["coordinates"]
    )
    for rings in polygons:
        ax.add_patch(
            Polygon(
                np.asarray(rings[0]),
                closed=True,
                facecolor="#e5e7eb",
                edgecolor="#4b5563",
                linewidth=0.5,
            )
        )

mapped = df.loc[~df["missing_geocode"]]
ax.scatter(
    mapped["longitude"],
    mapped["latitude"],
    s=7,
    color="#dc2626",
    alpha=0.45,
    linewidths=0,
)
ax.set(
    xlim=(-74.28, -73.68),
    ylim=(40.48, 40.93),
    xlabel="Longitude",
    ylabel="Latitude",
    title="Crash locations, June 30-July 6, 2024",
)
ax.set_aspect("equal", adjustable="box")
fig.tight_layout()
plt.show()

```



4. Crash severity by borough

A crash is classified as severe when at least one person was injured or killed.

```
df["severe"] = (
    (df["number_of_persons_injured"] > 0)
    | (df["number_of_persons_killed"] > 0)
).astype("int8")

severity_table = pd.crosstab(df["borough"], df["severe"])
display(severity_table)

chi2, p_value, degrees_of_freedom, expected = chi2_contingency(severity_table)
print(f"Chi-square statistic: {chi2:.3f}")
print(f"Degrees of freedom: {degrees_of_freedom}")
print(f"P-value: {p_value:.4f}")
print(
    "Conclusion: "
    + (
        "reject the null hypothesis of independence."
        if p_value < 0.05
        else "do not reject the null hypothesis of independence."
    )
)
```

```
)
)
```

	severe	0	1
borough			
BRONX		100	89
BROOKLYN		240	176
MANHATTAN		108	105
QUEENS		179	157
STATEN ISLAND		33	12

Chi-square statistic: 9.656
 Degrees of freedom: 4
 P-value: 0.0466
 Conclusion: reject the null hypothesis of independence.

5. Merge with the NYC ZIP-code reference

The earlier analysis incorrectly joined ZIP codes to NYPD precinct numbers. Here each ZIP code is joined to the official modified ZCTA and its population estimate with a many-to-one validation.

```
zip_reference = pd.read_csv(
    DATA_DIR / "nyc_modified_zip_codes.csv",
    dtype={"modzcta": "string", "label": "string", "zcta": "string"},
)
zip_lookup = (
    zip_reference.assign(zip_code=zip_reference["label"].str.split(r",\s*"))
    .explode("zip_code")
    .loc[:, ["zip_code", "modzcta", "pop_est"]]
    .dropna(subset=["zip_code"])
    .drop_duplicates("zip_code")
)

df["zip_code"] = df["zip_code"].str.extract(r"(\d{5})", expand=False)
merged_data = df.merge(zip_lookup, on="zip_code", how="left", validate="m:1")

known_zip = merged_data["zip_code"].notna()
match_rate = merged_data.loc[known_zip, "modzcta"].notna().mean()
print(f"Rows preserved by left join: {len(merged_data) == len(df)}")
print(f"Reference match rate among known ZIP codes: {match_rate:.1%}")
display(merged_data[["zip_code", "modzcta", "pop_est"]].head(10))
```

Rows preserved by left join: True
 Reference match rate among known ZIP codes: 99.8%

	zip_code	modzcta	pop_est
0	<NA>	<NA>	NaN
1	10455	10455	42283.0
2	10004	10004	3028.0
3	<NA>	<NA>	NaN
4	<NA>	<NA>	NaN
5	<NA>	<NA>	NaN
6	11207	11207	91972.0
7	11208	11208	98660.0
8	11432	11432	62053.0
9	<NA>	<NA>	NaN